

面向高频 DApp 的 Hybrid TEE-Rollup 可恢复挑战协议与轻量 DA 成本评估

Recoverable Challenge Protocol and Evidence-Carrying DA Commitments for Hybrid TEE-Rollups in High-Frequency DApps

作者：杨帆

摘要

高频 DApp，如元宇宙状态同步、去中心化社交交互和链上 AI 推理服务，把扩容问题转化为一个 systems/security tradeoff：系统既要保持低交互成本和低延迟，又不能在异常时放弃公开可验证性和挑战活性。全链上执行提供强公开验证，但难以承受大量小额、连续状态更新；完全链下执行降低成本，却把异常仲裁交给不可公开审计的服务边界；单纯依赖 TEE 又会引入硬件信任、宿主可用性和远程证明边界。本文研究 Hybrid TEE-Rollup 中一个窄但关键的问题：当执行者只向链上提交轻量承诺、完整 payload 位于 DA 层，而 challenge 又因 timeout 停滞时，协议如何恢复争议、重放证据并完成仲裁。核心主张不是生产级 Rollup 完整实现，而是三个可组合的机制接口：recoverable challenge、evidence-carrying compact commit 和 DA-aware cost model。基于 Python prototype、本地 EVM gasUsed 和 Sepolia 部署记录，实验显示 compact commit 平均约 406 bytes；在给定 cost model 与 DA profile 下，compact modular DA sampling 相比 full-onchain calldata 在 payload=8192、prompt length=64、batch size=1000 时得到 96.33% 的配置化估算降本；recoverable challenge 在 timeout 场景下将 challenge success 从 0% 提升至 100%。这些结果说明机制接口、活性差异和成本趋势，不外推为真实 TEE 安全、生产级 DA 或主网费用结论。

关键词：Hybrid TEE-Rollup；交互式挑战；数据可用性；高频 DApp；成本摊销；故障恢复

1 Introduction

高频 DApp 的困难不只是吞吐量。单次交互通常价值不高，却要求持续状态更新、低延迟反馈、低链上负担和异常时的公开可验证仲裁。以链上 AI 推理服务为例，用户连续提交 prompt，执行者在 TEE 中生成 response，并向链上提交 compact commit；完整 prompt、response 与 attestation 留在 DA 层。正常路径希望尽量轻，但异常路径必须足够重：如果 response 与 DA payload 不一致，或 challenge 在中途 timeout，系统不能只报告“异常已检测”，还必须保证争议能继续推进到 replay 与 resolve。

现有路线各自解决了 tension 的一部分，却没有同时消除四个约束。Fully on-chain execution 保留公开验证，但把高频 payload、存储和执行成本直接推到链上；fully off-chain execution 降低延迟和成本，却缺少用户可依赖的公开仲裁路径；pure TEE execution 可以降低正常路径验证成本，但把安全和可用性压在硬件、宿主 I/O、远程证明和服务连续性上。Hybrid

TEE-Rollup 的吸引力正在于它把 TEE 快速路径与链上异常路径组合起来；其风险也在这里：异常路径如果缺少恢复和证据入口，系统会在最需要仲裁时失去协议连续性。

timeout 后 challenge 卡死不是普通工程异常，而是 protocol liveness 和 arbitration continuity 问题。传统 challenge 机制可以强调错误能否被发现，但对于高频 DApp，检测只是争议处理的入口：未完成的 challenge 会占用状态、延迟结算，并让错误提交停留在不可结算的中间区间。本文因此把 failure → recovery → replay → resolution 作为主线。异常被观察后，协议需要恢复会话；恢复后，compact commit 必须能指向 replay 所需证据；replay 后，仲裁结果才可能被 resolve 或 slash。

轻量提交也有安全语义边界。compact commit 如果只是普通 hash，确实能压缩链上字节，却不能告诉验证者如何取回 payload、验证字段级 membership、或为 replay 构造 evidence context。本文将 compact commit 定义为 minimal verifiable evidence entry：它在正常路径保持提交小，在异常路径仍能回指 DA payload、Merkle proof、attestation digest 和执行摘要。实验部分采用 synthetic workload、simulated TEE、mock/verifiable DA registry、本地 EVM gasUsed 和 Sepolia deployability evidence；这些证据用于评估机制接口和趋势，而不是声称生产网络性能。

1.1 Contributions（本文贡献）

Recoverable Challenge. The first contribution is a liveness-preserving challenge transition for Hybrid TEE-Rollups. Timeout can leave a dispute unresolved even when the system has already detected abnormal behavior; this is especially harmful for high-frequency DApps, where many small interactions depend on continuous settlement rather than one-off manual recovery. The proposed recover transition does not change correctness facts, expected traces, or claimed traces. Its significance is narrower and more precise: it turns a stalled challenge session back into a replayable and resolvable arbitration path.

Evidence-Carrying Compact Commit. The second contribution is a compact commitment that is replay-ready, DA-traceable, and verification-oriented. Unlike a plain hash, the commit binds state root, output hash, proof hash, DA pointer, and DA root in one context. This makes the on-chain object a minimal evidence entry: small enough for the normal path, but structured enough for a challenger to retrieve DA evidence and replay disputed computation when the exceptional path is activated.

DA-aware Cost Model. The third contribution is a lightweight model for reasoning about payload scaling, batch amortization, and DA route selection under high-frequency workloads. The model is not a pricing oracle. Its role is to help system designers understand when full calldata becomes structurally unfavorable, when batch

amortization matters, and how replay-ready DA evidence changes the cost boundary of a compact submission design.

表 1 本文机制推进点与研究作用

机制推进	核心作用	论文定位
Recoverable Challenge	在 timeout 或中断后恢复争议流程	提升 challenge liveness
Evidence-Carrying Compact Commit	绑定 state/output/proof/DA root	形成最小证据入口
Fault Taxonomy	区分 attestation、DA、timeout、replay 和 trace 故障	明确异常责任边界
DA-aware Cost Amortization	联合 payload、batch 和 DA route 分析成本	支撑高频 DApp 成本论证

表 2 Mechanism-Oriented Comparison (机制语义对比)

表 2 helps the reviewer place the mechanism rather than read it as a capability checklist: existing systems provide important challenge or TEE fast-path components, while this work isolates a narrower gap – timeout-resilient challenge liveness, commit-to-DA evidence binding, and cost reasoning for replay-ready high-frequency workloads.

System	Challenge role	Recovery semantics	DA evidence binding	Cost model role
Optimism	Fraud proof / dispute game	Timeout handled by challenge window assumptions	Not the focus of this comparison	Not modeled here
Arbitrum	Interactive dispute localization	Protocol-specific dispute progression	Not the focus of this comparison	Not modeled here
OTR	TEE-assisted optimistic verification	Limited timeout recovery discussion	Partial payload/proof linkage	Not modeled here
TEERollup	TEE fast path with challenge entry	Discussed mainly as system mechanism	Discussed at architecture level	No lightweight DA amortization model
This Work	Recoverable	Recover as	Commit-to-DA	Payload/batch/route

	challenge state machine	liveness-preserving transition	field proof linkage	tradeoff analysis
--	-------------------------	--------------------------------	---------------------	-------------------

2 Background and Threat Model

2.1 Rollups, TEE, and Data Availability

Rollup 的基本思想是将大量执行过程移至链下完成，并将状态承诺、证明或争议入口提交到链上。ZK-Rollup 通过有效性证明保证提交状态的正确性，具有较强的验证属性，但证明生成、验证电路设计和复杂系统实现带来较高成本。Optimistic Rollup 假设链下执行结果默认正确，只有在挑战期内出现争议时才进入 fraud proof 或交互式验证流程，因此常规路径成本较低，但结算延迟和挑战机制复杂度成为关键限制。

TEE attestation 为链下执行可信性提供补充接口。可信执行环境可以在隔离环境中执行程序，并对输入、输出、环境标识和 nonce 生成证明，使外部验证者能够确认某一输出来自指定执行环境。然而，TEE 更适合作为降低正常路径验证成本的组件，而不应替代链上挑战、数据可用性和故障追责机制。

Optimistic challenge 是 Rollup 系统中处理异常的重要机制。典型流程包括提交状态承诺、开启挑战、交互式缩小争议范围，并在必要时执行单步重放或链上仲裁。对于长执行轨迹，二分定位能够将争议定位复杂度从线性扫描降低到对数轮次；但 challenge 本身也可能遭遇 timeout、应答缺失或证据不可用，因此其活性和恢复性需要被纳入协议设计。

Data Availability 关注链下或链外数据是否可被需要的验证者获得。模块化区块链和 LazyLedger 等路线强调将数据可用性从执行验证中解耦，使共识层主要负责数据排序和可用性保证。对于 Hybrid TEE-Rollup，DA 的作用不仅是降低存储成本，还要支撑后续 challenge 的证据回溯。因此，compact commit 必须与 DA proof 建立可验证联系。

2.2 Problem Scope and Adversarial Assumptions

This section narrows the problem after the background: the paper asks how a Hybrid TEE-Rollup can keep the common path cheap while preserving a public, recoverable, evidence-backed exceptional path. The cost dimension requires submissions that do not grow linearly with full payload size; the verifiability dimension requires compact commits that point to checkable DA evidence; the liveness dimension requires timeout not to permanently halt arbitration.

攻击面覆盖提交者、执行环境、DA 绑定和挑战流程。本文假设执行者或提交者可能提交错误 response、篡改 DA payload、提供无效 attestation，或在挑战流程中延迟响应；DA 层可能出现 payload 缺失、Merkle proof 损坏或 root 不一致；挑战流程本身也可能因超时、中断或 trace

长度不一致而无法自然进入 resolve 阶段。验证者、挑战者或 watchdog 可以观察链上提交和 DA 证据，并在发现异常时发起或恢复挑战。

Scope. The threat model deliberately excludes real hardware side channels, remote-attestation supply-chain failures, real DA network economic security, mainnet MEV, and congestion-driven fee dynamics. This scope is not a production claim; it defines what the prototype evidence can support: protocol state continuity, evidence linkage, configuration-based cost trends, and contract-level correspondence.

3 System Overview

This section explains how the mechanisms compose end to end. The system has three roles: an execution layer that creates a fast-path response and attestation, a DA layer that stores replay evidence, and a verification layer that turns faults into challenge/recover/replay/resolve transitions. The running example remains the high-frequency AI inference workload: many low-value requests should not force full payload publication on chain, but a disputed request must still leave enough evidence for public arbitration.

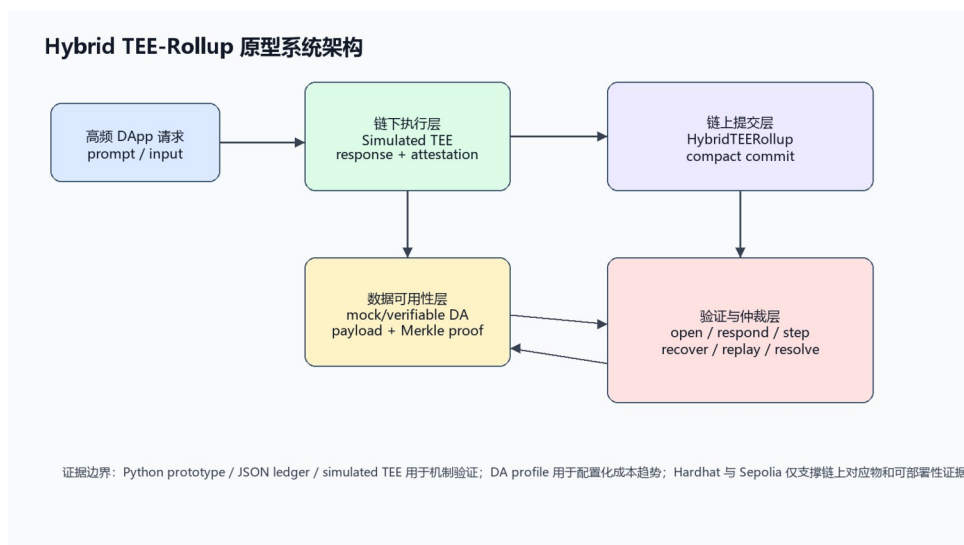


图 1 Hybrid TEE-Rollup 原型系统架构

如图 1 所示，系统由高频 DApp 请求入口、链下执行层、数据可用性层、链上提交层和验证仲裁层组成。正常路径中，执行层输出 response 与 attestation，数据层保存完整证据，链上提交层仅保存 compact commit。异常路径中，验证仲裁层通过 DA pointer 取回证据，并以 challenge、recover、replay 和 resolve 推进状态。这个划分使低成本正常路径与可仲裁异常路径保持同一证据上下文。

3.1 Execution Layer and Normal Path

执行层负责处理高频 DApp 请求。当前原型采用 deterministic mock model 或可选模型后端生成响应，并使用 simulated TEE attestation 对输入哈希、输出哈希、nonce 和执行环境标识进行绑定。attestation 的作用是为链下执行结果提供最小可信摘要，使后续验证层能够检查提交结果是否与声明的执行过程一致。

在系统叙事中，simulated TEE 的角色是建立“链下执行摘要 - 链上提交 - 异常挑战”的接口关系。真实 TEE 安全性不在本节展开，而在 Discussion and Limitations 中集中限定。

3.2 Data Availability Interface

The DA interface is where compactness becomes security-relevant. The data layer stores the full payload and verifiable structure - prompt, response, attestation, payload hash, leaf hashes, Merkle root, and field-level Merkle proof - while the chain stores only the compact commitment. The section therefore separates byte reduction from evidence preservation: the normal path avoids full payload publication, but the exceptional path can still recover the payload context needed for replay.

在运行示例中，如果执行者提交的 response 被替换，验证者可从 DA entry 取回 response 字段和对应 Merkle proof，并与链上 da_merkle_root 检查一致性；如果 payload 缺失，则异常停留在 DA availability fault，而不是被误写成已完成 replay 的执行错误。

3.3 Verification Layer and Recovery Trigger

验证层负责处理异常路径。当前原型实现了 challenge open、respond、step、recover、replay 和 resolve 等状态推进。其目标不是将所有执行都搬到链上，而是在出现争议时，通过证据加载、二分定位、单步重放和仲裁结算，将异常状态推进为可解释的结果。

验证层同时承担 fault taxonomy 的职责。本文将故障细分为 attestation fault、DA fault、timeout fault、replay fault 和 trace inconsistency。不同故障对应不同的观察和处理路径。例如，DA unavailable 表明数据层无法提供 payload；DA proof invalid 表明 payload 与链上 root 不一致；trace length mismatch 表明执行轨迹与声明结构不一致；response tampering 则可能同时导致 payload hash、attestation 和 state root mismatch。

在 Solidity 映射中，原型包含 MockTEEVERifier、MockDARegistry、MerkleVerifier 和 HybridTEERollup 等合约，用于表达链上状态推进和最小验证逻辑。本地 EVM 合约并非 Python 原型的逐行移植，而是抽取适合链上表达的提交、验证、挑战、恢复和结算路径。

4 Recoverable Challenge Protocol

4.1 Protocol Model and Definitions

This section defines the protocol boundary that makes recovery safe to discuss. A recover operation is useful only if it restores progress without mutating the dispute

facts. The definitions below separate session state from Challenge Facts so that timeout recovery can be treated as a liveness transition rather than a hidden correctness decision.

Definition 1 (Challenge Session)

挑战会话定义为: $cs = (id, s, F, T)$ 。

其中, id 表示会话标识; s 表示当前状态; F 表示 Challenge Facts (挑战事实集合); T 表示超时参数。

Definition 2 (Challenge State Machine)

挑战状态机定义为: $M = (S, E, \delta)$ 。

其中, S 为状态集合; E 为事件集合; $\delta: S \times E \rightarrow S$ 为状态转移函数。

Definition 3 (Recovery Function)

恢复函数定义为: $Recover(cs) = cs'$ 。

恢复仅改变活性相关状态, 不改变 Challenge Facts, 即满足: $F(cs) = F(Recover(cs))$ 。

Protocol Property

基于上述定义, `recover` 是活性层变换而非正确性裁决: `Recover` 允许会话从 `TIMED_OUT` 等停滞状态回到可推进状态, 但不重写 `commit`、`DA root`、`payload hash`、`trace` 或 `mismatch evidence`。该边界把恢复语义与 `slashing` 语义分开, 也解释了为什么恢复不会改变 `correctness`: 后续 `replay` 仍然基于原始 Challenge Facts。

4.2 Timeout, Recovery, Replay, and Resolution

Recoverable Challenge Protocol 的目标, 是将 `challenge` 从一次性错误检测流程扩展为异常场景下可恢复推进的协议状态机。其基本状态包括 `PENDING`、`OPEN`、`RESPONDED`、`NARROWING`、`RECOVERED`、`READY_FOR_REPLAY`、`REPLAYED`、`RESOLVED`、`SLASHED` 和 `FINALIZED`。

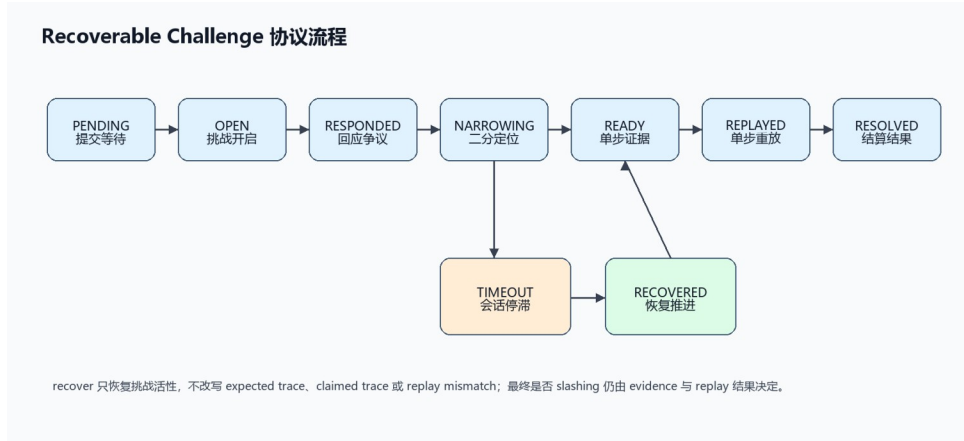


图 2 Recoverable Challenge 协议流程

当交易处于 PENDING 状态时，挑战者可以调用 open 进入 OPEN 状态。系统加载 compact commit、DA status 和相关 mismatch evidence。随后，被挑战方进入 respond 阶段，提供其对争议的回答。若争议涉及执行轨迹，协议进入 step 阶段，通过 bisection narrowing 缩小争议区间。对于长度为 n 的 trace，理想情况下定位轮次接近 $\log_2(n)$ ，从而避免线性扫描长执行轨迹。

当争议区间被缩小到单步时，协议进入 single-step replay。replay 阶段比较 expected hash 与 claimed hash，判断局部执行是否一致。如果重放失败，resolve 阶段可以将交易标记为 SLASHED；如果重放通过，则挑战不成立或进入相应结算逻辑。

recover is the key transition because it repairs protocol progress without changing the evidence. In a non-recoverable challenge, timeout can leave the session in a state where the dispute is known but no replay or resolution follows. Recoverable challenge adds a watchdog/operator transition that moves the timed-out session back to a state where step, replay, and resolve can continue. The distinction matters because liveness and correctness are different obligations: recover restores the former while preserving the latter for replay.

协议流程可分为三条路径。第一是正常路径：若挑战期内没有有效争议，交易最终进入 FINALIZED。第二是可仲裁异常路径：若 response tampered 或 trace length mismatch 等执行相关故障出现，协议通过 open、respond、step、replay 与 resolve 完成争议定位和结算。第三是 timeout 恢复路径：若挑战过程因超时中断，watchdog 或 operator 可以调用 recover 将会话推进到 RECOVERED，并继续后续 replay 与 resolve。该划分避免将“异常检测”“活性恢复”和“经济惩罚”混写为同一性质。

4.3 Correctness Boundary and Protocol Properties

性质一，DA proof 绑定性质。compact commit 中的 da_merkle_root 与 DA entry 中的字段级 Merkle proof 绑定，因此 prompt、response 或 attestation 字段被替换后，会导致

payload hash、leaf hash 或 Merkle proof 与链上 root 不一致。该性质不依赖真实 DA 网络，只说明当前 verifiable DA 结构能检测字段篡改。

性质二，recover 的活性性质。recover 操作的目标是恢复超时挑战会话，使其继续进入 step、replay 和 resolve；它不修改 expected trace、claimed trace 或 replay mismatch 的事实。因此，recover 不改变争议判断本身，而是避免 timeout 让争议流程永久停在中间状态。

性质三，二分定位性质。对于长度为 n 的执行 trace，challenge-step 每轮缩小争议区间，理想轮次接近 $\log_2(n)$ 。本文实验中 trace steps 为 4、8、16、32、64、128 时，平均二分轮次分别为 2、3、4、5、6、7，与理论对数趋势一致。

5 Evidence-Carrying Compact Commit and DA Binding

This section answers why the compact commitment is not merely a compression artifact. Its research value is that it carries the minimum evidence linkage needed by the exceptional path. The commit binds state_root, output_hash, proof_hash, da_pointer, and da_merkle_root: state_root constrains input/output/environment summaries; output_hash constrains response; proof_hash constrains attestation; da_pointer locates the full payload; da_merkle_root commits to field-level proof membership.

这种绑定关系使验证者能够从链上提交进入 DA 证据结构。若 response 被篡改，验证者检查 response leaf 与 Merkle root；若 attestation 与输出不一致，proof_hash 和 payload hash 形成交叉约束；若 DA proof 损坏，系统可把异常定位为 DA proof invalid，而不是混同为执行重放失败。由此，compact commit 支持后续 challenge/replay 的证据加载，而不是只服务于链上字节压缩。

Adversarially, a submitter may replace payload fields, hide a DA entry, submit an output inconsistent with attestation, or delay during dispute. The compact commitment does not solve real DA availability or real TEE trust by itself. Its narrower contribution is to define what the chain-facing object must bind so that a later challenge is replay-ready rather than merely byte-efficient.

6 DA-aware Cost Model

成本模型关注 full-onchain 与 compact commit + DA 在高频场景下的结构性差异。full-onchain 路线将完整 payload 作为链上数据提交，其成本随 payload size 增大近似线性增长。compact commit 路线只将固定摘要结构提交到链上，将完整 payload 放入 DA 层，并通过 DA root 和 Merkle proof 保留验证能力。

The model compares four DA routes - full-onchain calldata, compact external DA, compact EIP-4844-like, and compact modular DA sampling - as configuration-based estimates. It should be read as a systems tradeoff model, not as real-world pricing.

Payload size determines when full publication dominates; batch size determines whether fixed commitment overhead is amortized; DA route determines the marginal cost and dependency introduced by moving replay evidence off calldata.

模型表达为 $C_{total} = C_{commit} + C_{DA} + p_{challenge} * C_{challenge}$ 。其中 C_{commit} 表示链上 compact commit 或 full payload 提交成本, C_{DA} 表示不同 DA route 下的数据成本, $C_{challenge}$ 表示发生争议时的挑战、恢复、重放和结算成本, $p_{challenge}$ 表示争议发生概率。批处理后单笔摊销成本为 $C_{amortized} = C_{total} / batch_size$ 。当前实验主要比较不同 route 在相同参数下的结构性趋势, 而不估计真实业务中的 $p_{challenge}$ 。

在系统解释上, batch size 越大, 固定提交成本越容易被摊薄; payload 越大, full-onchain calldata 的线性成本越突出; DA route 越接近低边际数据成本, compact commit + DA 的收益越明显。当前 Python prototype 中的成本实验是配置化估算模型, 本地 EVM gasUsed 则用于补充链上状态推进基线, 两者在 Evaluation 中明确区分。

7 Evaluation

Evaluation is organized around implication-driven comparisons rather than prototype validation alone. RQ1 asks whether compact commit isolates payload growth while preserving evidence linkage. RQ2 asks how batch amortization and DA-route selection change the feasibility boundary for high-frequency workloads. RQ3 asks whether recovery converts timeout detection into arbitration completion. RQ4 asks which faults are only detectable and which are replayable/slashable. RQ5 asks whether the protocol has a contract-level counterpart. All workloads are synthetic; cost numbers are configuration-based estimates or local EVM gasUsed baselines; Sepolia evidence is deployability evidence, not production performance.

本文成本实验使用如下抽象模型:

$$C_{total} = C_{commit} + C_{DA} + p_{challenge} * C_{challenge}$$

$$C_{amortized} = C_{total} / batch_size$$

Here C_{commit} captures the configured chain-facing submission cost, C_{DA} captures the configured data-publication profile, $C_{challenge}$ captures challenge/recover/replay/resolve cost, and $p_{challenge}$ denotes dispute probability. The experiment does not estimate real $p_{challenge}$, blob market price, DA network fee, or production transaction fee. The model is used to compare structural trends under controlled synthetic inputs.

7.1 RQ1: Does compact commit reduce on-chain submission size?

RQ1 isolates the submission structure. The Python prototype generates synthetic ledger entries under a simulated TEE and compares canonical JSON bytes for full payload and compact commit. The purpose is not to predict calldata price, but to test whether payload growth is separated from the on-chain object while DA roots and proof pointers still preserve a replay path.

度量指标包括 `full_bytes`、`compact_bytes` 和 $\text{byte_reduction_ratio} = (\text{full_bytes} - \text{compact_bytes}) / \text{full_bytes}$ 。`full_bytes` 表示完整 `prompt`、`response`、`attestation` 及相关字段都作为 payload 提交时的字节规模；`compact_bytes` 则只统计 `state_root`、`output_hash`、`proof_hash`、`da_pointer` 与 `da_merkle_root` 等摘要字段。

The comparison shows the separation clearly: compact commit stays near 406 bytes, while full payload bytes grow with payload size. At payload size 8192, full payload bytes reach about 50068.84 / 50262.84 for prompt length 64 / 256, whereas compact bytes remain fixed by the commitment schema. The result reflects the intended systems boundary: application payload is no longer carried by the chain-facing object, but the DA pointer and Merkle root retain the evidence linkage needed by the exceptional path.

The implication is limited but central. Compact commit is useful because it separates payload scaling from on-chain submission without discarding replay readiness. A design that only stores an unstructured hash would also be small, but would not provide the same DA-traceable path into prompt, response, attestation, and field-level proof evidence.

这一结论的解释边界是编码层面的：字节数来自 Python prototype 的 canonical JSON 表达，不能直接等价于真实链上 calldata、storage layout 或 ABI 编码后的完整成本；真实 attestation 与 DA 证明大小也会改变绝对数值。

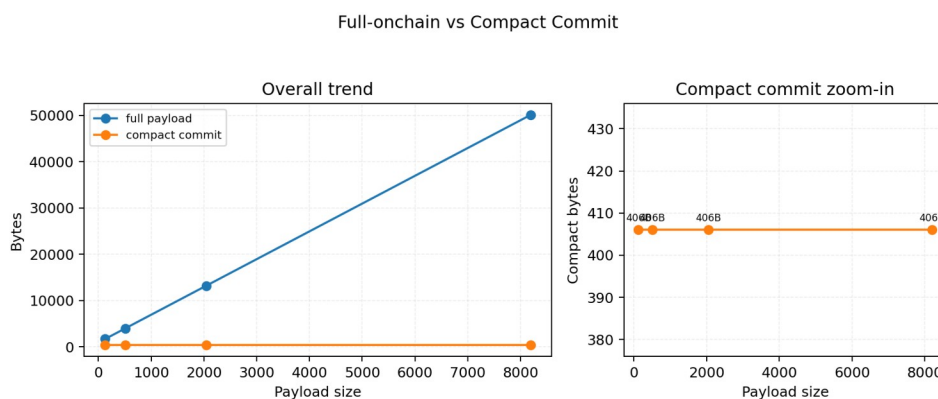


图 3 full payload 与 compact commit 的字节规模对比：payload growth 被隔离到 DA 侧，而 compact commit 保留 replay 所需的证据入口

7.2 RQ2: How does batch size affect DA-aware amortized cost?

RQ2 asks when the submission strategy changes system feasibility under high-frequency synthetic workloads. The experiment compares `full_onchain_calldata`, `compact_external_da`, `compact_eip4844_like`, and `compact_modular_da_sampling` across payload size, prompt length, and batch size. The reported amortized gas values are configuration-based estimates from script profiles, not live network fees.

主要指标为 $\text{amortized_gas} = \text{total_gas} / \text{batch_size}$ 和 $\text{reduction_ratio} = (\text{full_onchain_gas} - \text{target_gas}) / \text{full_onchain_gas}$ 。 `total_gas` 来自配置化 profile，而不是主网交易回执；因此这里讨论的是结构性趋势，而不是实时网络价格。

The trend suggests that batch amortization becomes increasingly important once payload publication dominates submission overhead. For `payload=8192` and `prompt length=64`, `full_onchain_calldata` falls from 801101.44 estimated gas at `batch size=1` to 801.10 at `batch size=1000`, while `compact_modular_da_sampling` falls from 29026.98 to 29.03. The 96.33% configured reduction at `batch size=1000` reflects a structural advantage of moving large replay evidence to a lower-cost DA profile under high batch amortization; it is not a mainnet fee claim.

The system insight is that payload scaling and batch size interact. Full calldata deteriorates when payload dominates because every larger response increases chain-facing bytes. Compact DA routes become more attractive when many interactions can share fixed submission overhead, but they also introduce DA dependency and proof-retrieval assumptions. This is the tradeoff a high-frequency DApp designer must evaluate, not simply a question of which route is cheapest in one configuration.

96.33% 是给定 cost model 与 DA profile 参数下的趋势结果。它说明 compact modular DA sampling 在大 payload 与大 batch 条件下具有结构性优势，但真实公网费用仍会受到 calldata 定价、blob 价格、交易拥堵、DA 网络实现和业务争议率影响。

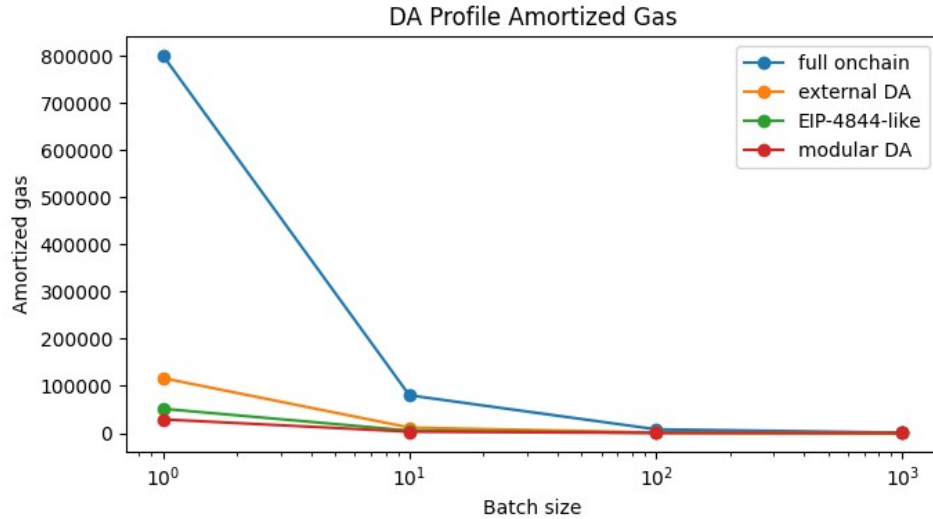


图 4 不同 DA profile 下的摊销 gas 趋势：当 payload publication 成为主要开销时，batch amortization 和 DA route selection 共同决定结构性成本边界

7.3 RQ3: Can recovery improve challenge liveness under timeout?

RQ3 tests the central liveness claim. The Python prototype uses the same synthetic challenge state machine for no-recover and recoverable paths, with trace steps 4, 8, 16, 32, 64, 128, challenge timeout 2 seconds, and max bisection rounds 16. The question is whether timeout detection becomes arbitration completion: can the session still reach replay, resolve, and slashing after it stalls?

这里记录 `avg_bisection_rounds`、`avg_expected_log2_rounds`、`recovery_success_rate`、`challenge_success_rate` 和 `slashed_rate`。 `avg_bisection_rounds` 用来检查争议定位复杂度是否接近理论预期， `challenge_success_rate` 与 `slashed_rate` 则反映仲裁是否真正走到完成状态。

The bisection behavior matches the expected logarithmic trend, but the more important comparison is no-recover versus recover. In `challenge_timeout_no_recover`, detection reaches 100% while challenge success and slashing remain 0%; the session is known to be abnormal but remains unresolved. In `challenge_timeout_recover`, detection, recovery, challenge success, and slashing all reach 100% under the synthetic scenario. The comparison indicates that timeout detection is not equivalent to protocol progress.

The implication is that recover changes progress continuity, not correctness. It does not rewrite expected trace, claimed trace, or replay mismatch. It reopens the path by which those facts can be tested and resolved. For high-frequency DApps, this

distinction matters because many small unsettled disputes can become a system-level availability problem even if every individual timeout is detectable.

当前实验中的 challenge trace、expected hash、claimed hash 和部分 replay evidence 由原型逻辑或链下流程提供。因此，RQ3 支撑的是 recoverable state machine 的活性主张，而不是完整链上 fraud proof、FPVM 或真实 ML/TEE 执行验证的安全性。

7.3.1 Recovery Overhead Analysis（恢复开销分析）

利用表 4（原表 3）中已有本地 EVM 数据，challenge step 的平均 gasUsed 为 93807，challenge recover 为 156891。由此可得恢复相对单轮 challenge step 的开销比约为：Recovery Overhead = $156891 / 93807 \approx 1.67$ 。

Recover costs more than one challenge step in the local EVM baseline, but it is paid on the exceptional path. The relevant comparison is not normal-path throughput versus recover cost; it is unresolved arbitration versus a bounded recovery transition. Under the local Hardhat measurement, the 1.67x ratio describes relative contract-path gasUsed only, not mainnet pricing or production fee behavior.

上述数值来自 Hardhat 本地 EVM 原型测量，用于解释异常路径的相对成本；主网 gas 价格、拥堵条件和生产级合约优化不在该测量范围内。

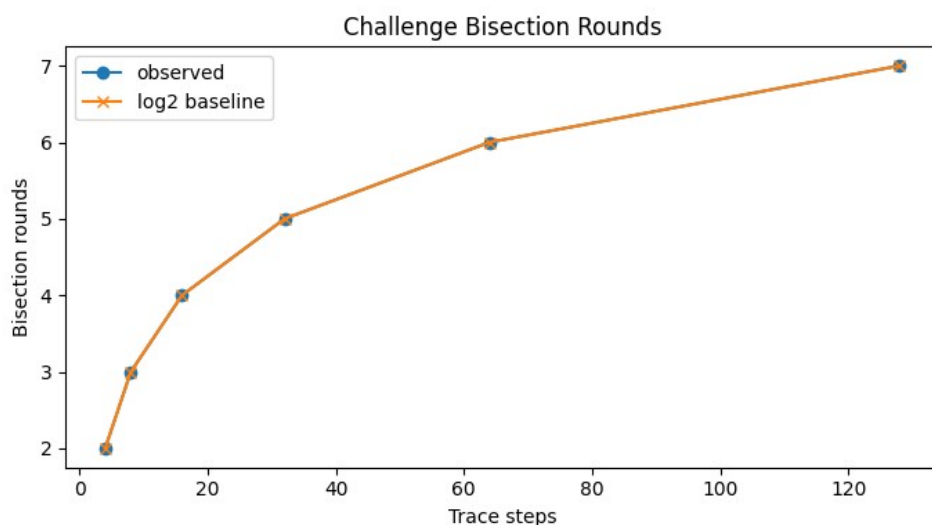


图 5 二分挑战轮次与 trace steps 的关系

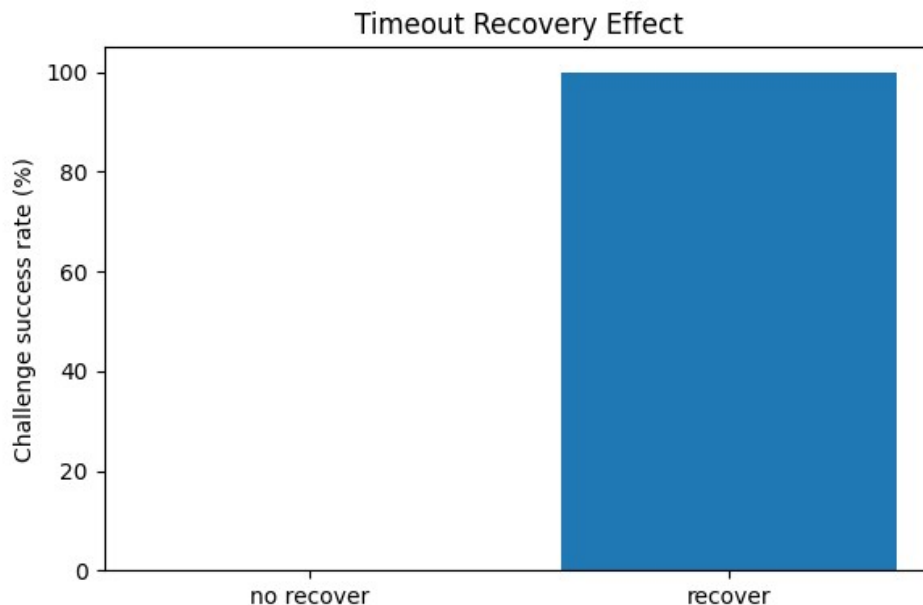


图 6 timeout recover 对挑战完成率的影响: no-recover 能检测 timeout, 但 recoverable challenge 才能把争议推进到 replay/resolve

7.4 RQ4: Which failure scenarios lead to detection, recovery, or slashing?

RQ4 makes the security evaluation fault-oriented. The experiment injects eight synthetic scenarios and asks which ones are detectable, recoverable, replayable, or slashable in the current prototype. This avoids a common overclaim: different faults should not be forced into the same slashing semantics simply because they can be detected.

指标为 `detection_rate`、`challenge_success_rate` 和 `slashed_rate`。为了避免把语义不同的异常混在一起, 实验分析分为四类: Normal path, Detectable but not unified slashing, Challenge + slashing, 以及 Timeout liveness comparison。

The comparison separates four outcomes. Normal path samples do not trigger false slashing. Attestation invalid, DA unavailable, and DA proof invalid are detectable but not mapped to unified slashing in the prototype. Response tampering and trace length mismatch can enter challenge + replay + slashing because comparable execution traces exist. Timeout only becomes slashable in the recoverable path, where the session can proceed after recovery. The result reflects a security boundary, not a weakness to hide: detection, recovery, replay, and punishment are distinct protocol capabilities.

RQ4 的重点不是宣称所有 fault 已经闭环, 而是明确当前闭环在哪里。响应篡改、trace 不一致和 timeout recovery 已形成较完整的 challenge + slashing 路径; attestation fault 与 DA fault 则主要体现为稳定检测、拒绝或报告能力。这个区分使 evaluation 更接近

systems/security 论文中的 threat-specific reasoning，而不是把“能发现异常”误写成“都能惩罚异常”。

后续仍需细化 fault taxonomy。特别是 DA unavailable、DA proof invalid 和 attestation invalid，应进一步区分何时直接拒绝、何时触发恢复、何时进入链上仲裁，以及何时应施加经济惩罚。

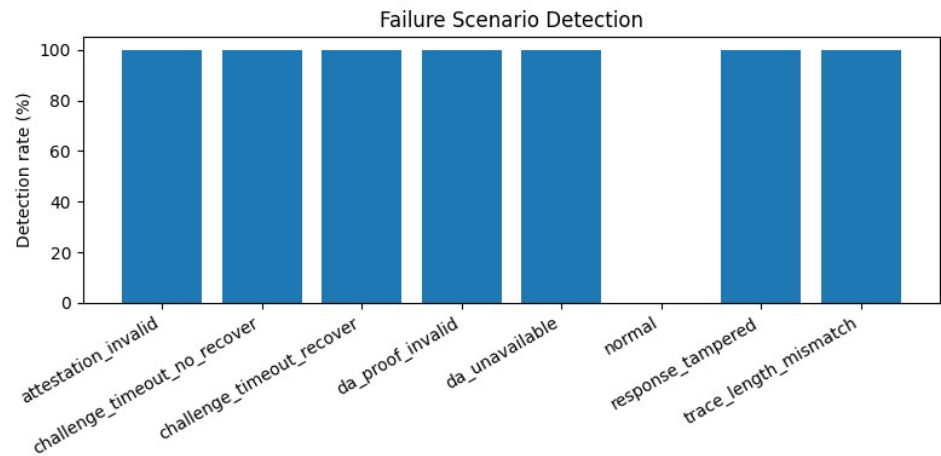


图 7 失败场景检测与仲裁结果：fault-oriented view 区分 detectable、recoverable、replayable 与 slashable，而不是把所有 fault 统一为同一种惩罚语义

表 3 失败场景分类与当前证据边界

类别	场景	当前结果	证据边界
Normal path	normal	不触发检测或 slashing	说明正常路径不误报
Detectable but not unified slashing	attestation_invalid, da_unavailable, da_proof_invalid	检测率 100%, challenge success/slashed 为 0%	说明可检测或拒绝，不说明已统一惩罚
Challenge + slashing	response_tampered, trace_length_mismatch	detection/challenge success/slashed 均为 100%	说明典型执行异常可进入闭环
Timeout liveness comparison	challenge_timeout_no_recover vs challenge timeout recover	no recover 为 0%, recover 后为 100%	说明 recover 改善仲裁活性

7.5 RQ5: Do Solidity, local EVM, and Sepolia evidence support an on-chain counterpart?

RQ5 asks whether the protocol design has a concrete contract-level counterpart. The Solidity prototype and Hardhat local chain measure key gasUsed paths – register DA, submit rollup, challenge open/respond/step/recover/replay/resolve, and finalize – while Sepolia deployment records provide chain ID, deployment block, addresses, and

ABI/address index. These artifacts support protocol-to-contract mapping and deployability evidence, not production readiness.

本地 EVM 指标是各关键函数调用的 gasUsed。Sepolia 证据包括 chain id、部署区块、合约地址和 ABI/address index。前者用于观察链上状态推进路径的基线开销，后者用于说明同一组核心合约已经具备公开测试网 deployability evidence。

本地 EVM 平均 gasUsed 包括：register DA 113526.00, submit rollup 299033.67, challenge open 283769.00, challenge respond 65886.00, challenge step 93807.00, challenge recover 156891.00, challenge replay 135632.00, challenge resolve 113402.00, finalize 33477.00。Sepolia 部署记录则给出了公开测试网链上地址与 ABI/address index。将这两类证据放在一起看，可以说明 Python prototype 中的提交、挑战、恢复、重放和结算路径已经被抽取成 Solidity 状态推进操作，但还不能说明它们已经经历生产网络压力测试。

The implication is a chain correspondence claim. Python state-machine operations are not merely pseudocode; they have been mapped into Solidity functions with local EVM gasUsed baselines and public testnet deployment artifacts. The claim remains deliberately narrow: it supports deployability and contract-path correspondence, not mainnet performance, economic security, or audit readiness.

本地 EVM gasUsed 不能等价于公网费用，Sepolia 部署也不能说明生产可用性、主网性能或经济安全。当前 Sepolia 证据仅支持 deployability evidence。

表 4 本地 EVM 关键路径平均 gasUsed：这些数值是 local EVM contract-path baselines，不代表 production network fee

操作	平均 gasUsed	解释
register DA	113526.00	MockDARegistry 记录 payload hash 与 DA root
submit rollup	299033.67	compact commit 提交路径
challenge open	283769.00	打开争议并加载证据
challenge respond	65886.00	被挑战方响应
challenge step	93807.00	二分定位的单轮推进
challenge recover	156891.00	恢复超时挑战会话
challenge replay	135632.00	单步重放仲裁
challenge resolve	113402.00	结算争议结果
finalize	33477.00	挑战期后最终确认

表 5 Sepolia 部署信息：该证据支持 deployability 和 protocol-to-contract mapping，不支持生产性能主张

项目	数值
Network / Chain ID	Sepolia / 11155111

项目	数值
部署区块	10825904
MockTEEVerifier	0xC42CB0Cf0D112Bd59E0f212F2DB2002ca50a8b6A
MockDARegistry	0xc42C64a7De05bf3ad4f1c75CbD8E0506F7De99Ff
HybridTEERollup	0xB9B72f10bB8aBC1ed090f1B0443Fe57189c497Fd

7.6 Reproducibility and Evidence Types

Python 原型位于 `project_code/tee_rollup_demo`，正式实验脚本为 `project_code/scripts/run_paper_experiments.py`，输出 `cost_grid.csv`、`challenge_grid.csv`、`failure_scenarios.csv`、`summary.json` 和 `figures` 目录。本地 EVM gas 脚本为 `project_code/evm/scripts/measure_gas.js`，输出 `paper_outputs/evm_gas/measured_gas_report.md` 与 `measured_gas_summary.json`。Sepolia 部署记录位于 `project_code/evm/deployments/sepolia.json`。本次 revision 额外生成 `comparison_figures` 目录和 `comparison_experiments.md`，用于记录 baseline comparison、fault-oriented comparison 和 DA route tradeoff 的派生图表。

复现实验可使用以下命令：`python -m unittest discover -s tests -v`；`python scripts/run_paper_experiments.py --clean --samples 100 --payload-sizes 128,512,2048,8192 --batch-sizes 1,10,100,1000 --trace-steps 4,8,16,32,64,128`；在 `project_code/evm` 目录下运行 `npm.cmd test` 和 `npm.cmd run measure:gas`。上述命令分别对应机制验证、配置化成本趋势和本地 EVM 状态推进基线，不能相互替代。

7.7 Comparison-Oriented Evaluation

The preceding RQs validate the mechanisms; this subsection explains why the comparisons matter. A conference reviewer should be able to distinguish a working prototype from a mechanism that changes an outcome. The added comparisons therefore ask: what happens without recover, without compact evidence binding, under different fault classes, and across DA route assumptions?

7.7.1 Recover vs No-Recover: Liveness Ablation

Baseline. The no-recover baseline uses the same timeout workload but does not invoke watchdog recovery after the challenge session times out. Treatment. The recoverable challenge path invokes recover and then continues into replay and resolve. Both paths use the same synthetic trace construction, timeout configuration, and prototype state machine.

Scenario	Detection	Replay	Resolve	Slashing	Finalization / terminal	Avg recovery delay
no-recover	100%	0%	0%	0%	0%	N/A

recoverabl e challenge	100%	100%	100%	100%	100%	1.0s configured after timeout threshold
------------------------------	------	------	------	------	------	---

表 6 Recover vs no-recover comparison under the synthetic timeout workload

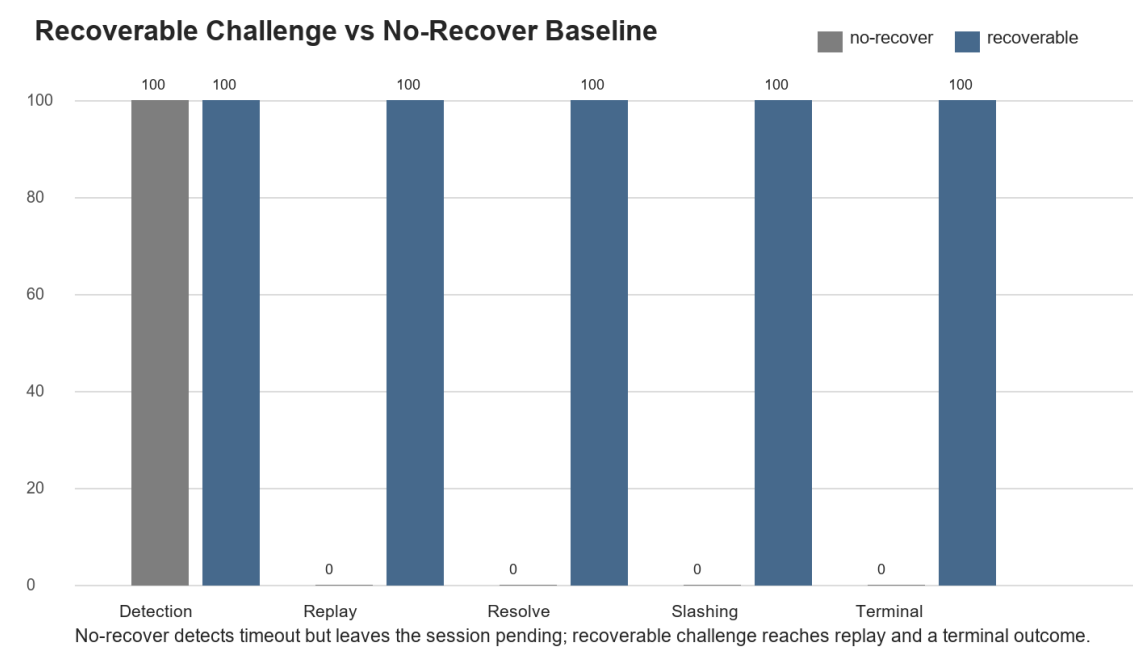


图 8 Recover vs no-recover across detection, replay, resolution, slashing, and terminal outcome rates

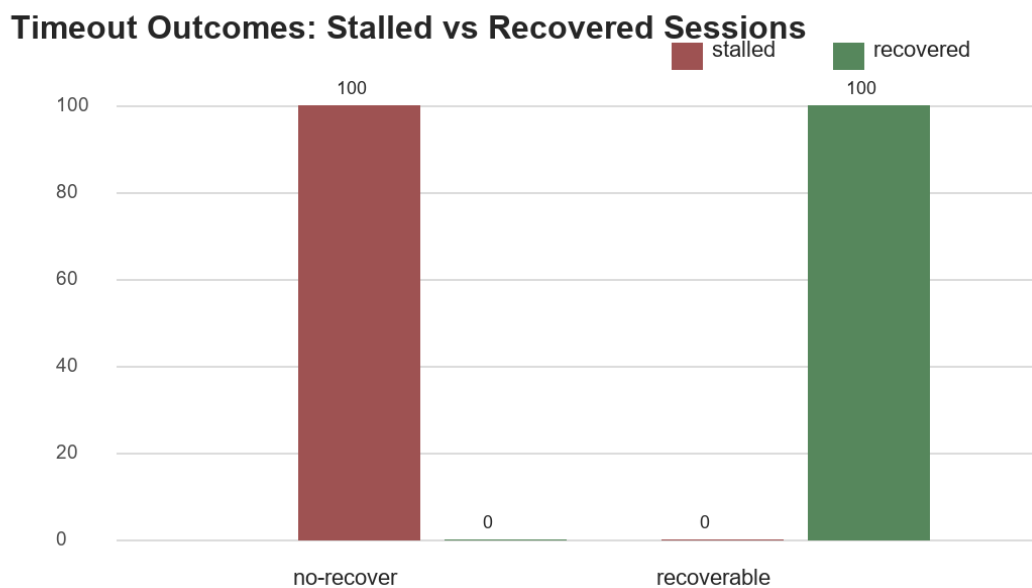


图 9 Timeout sessions remain stalled without recovery but reach recovered terminal outcomes with recover enabled

Replay/Recovery Timeline under Timeout

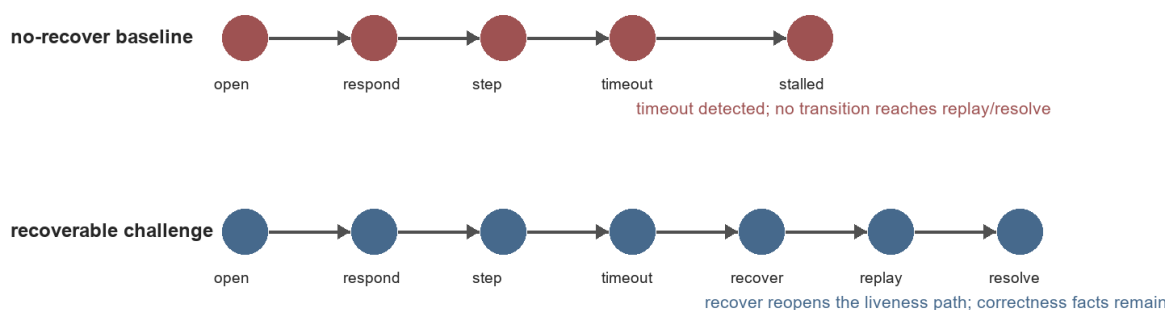


图 10 Replay/recovery timeline: recover changes liveness, while correctness facts remain unchanged

Observation. Both paths detect timeout, but only the recoverable path reaches replay and resolve. The comparison indicates that `detection_rate` alone is not a sufficient security metric for this protocol: timeout detection without a recovery transition

leaves the challenge session pending. The implication is that recover is a liveness-preserving transition, not an accuracy improvement or a correctness shortcut.

7.7.2 Compact Commit vs Full Payload: Evidence-Preserving Cost Comparison

The compact commit baseline comparison uses the same synthetic workload as RQ1. Full payload submission places prompt, response, attestation, and related fields in the on-chain payload. Compact commit stores fixed-size state/output/proof/DA-root fields and depends on DA retrieval for detailed replay evidence.

Scheme	On-chain bytes	Replay ready	Evidence retrieval	Verification cost
full payload	50,068.84 bytes at payload=8192, prompt=64	Yes; payload is directly present	No DA fetch dependency	High on-chain byte footprint
compact commit	406.00 bytes	Yes, if DA entry and proof are available	DA pointer + field Merkle proof	DA fetch + Merkle verification

表 7 Full payload and compact commit comparison

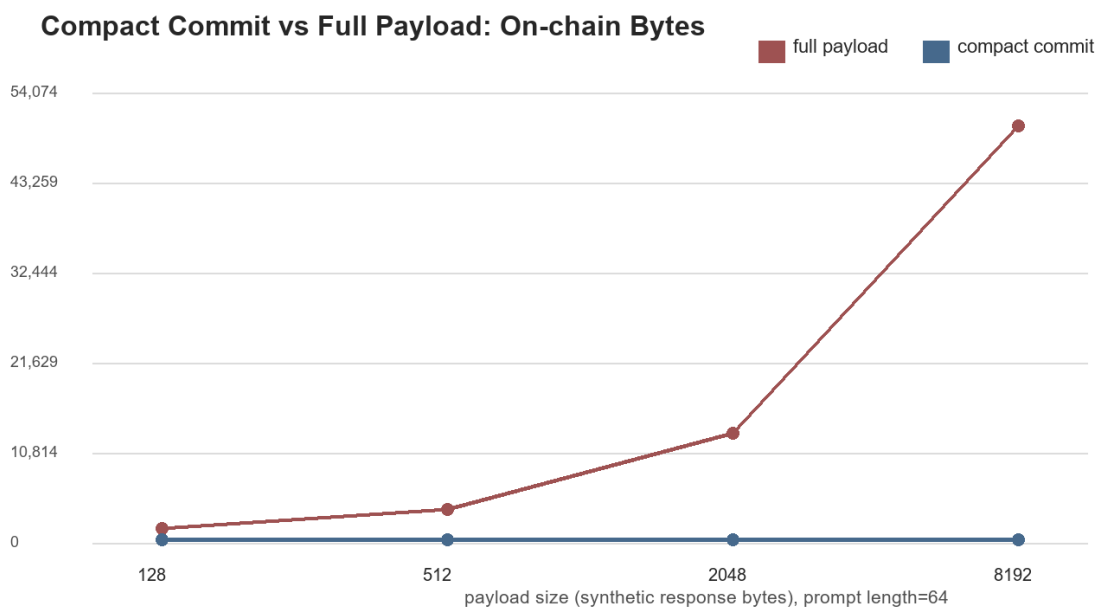


图 11 Full payload grows with synthetic payload size, while compact commit remains fixed-size in the prototype encoding

Observation. The result reflects more than compression. A plain hash would reduce bytes but would not specify how to retrieve the disputed response, verify field-level membership, or replay the relevant evidence. Compact commit matters because it keeps the common path small while preserving an evidence route for the exceptional path. The tradeoff is explicit: disputes pay DA fetch and Merkle verification overhead instead of forcing every normal interaction to publish full payload on chain.

7.7.3 Fault-Oriented Comparison

Security evaluation should not collapse all failures into a single success/failure outcome. The matrix below separates whether each injected fault is detectable, recoverable, replayable, and slashable in the current prototype. This is a prototype capability matrix, not a universal claim about production fault handling.

Fault Type	Detectable	Recoverable	Replayable	Slashable
response tampering	yes	no	yes	yes
DA unavailable	yes	no	no	no
DA proof invalid	yes	no	no	no
timeout / no recover	yes	no	no	no
timeout / recover	yes	yes	yes	yes
trace mismatch	yes	no	yes	yes
attestation invalid	yes	no	no	no

表 8 Fault-oriented comparison of detection, recovery, replay, and slashing

Fault-Oriented Comparison Matrix

	Detect	Recover	Replay	Slash
response tampering	yes	no	yes	yes
DA unavailable	yes	no	no	no
DA proof invalid	yes	no	no	no
timeout / no recover	yes	no	no	no
timeout / recover	yes	yes	yes	yes
trace mismatch	yes	no	yes	yes
attestation invalid	yes	no	no	no

The matrix separates detectable faults from faults the current prototype can recover, replay, and slash.

图 12 Fault-oriented comparison heatmap

Observation. Response tampering and trace mismatch can reach replay and slashing because the prototype can construct comparable expected and claimed traces. DA unavailable, DA proof invalid, and attestation invalid are detectable, but they are not yet mapped into a unified slashing path. The distinction matters because claiming that every detected fault is slashable would overstate the current system. The comparative view instead clarifies where the protocol has a closed arbitration path and where it still has a detect/reject boundary.

7.7.4 DA Route Tradeoff Comparison

The DA route comparison uses the existing configuration profiles: full_onchain_calldata, compact_external_da, compact_eip4844_like, and compact_modular_da_sampling. The figure fixes batch size at 1000 to show high-frequency amortization behavior and varies payload size. Values are gas-equivalent estimates from the script configuration, not live network prices.

Route	Amortized cost role	Replay readiness	Payload scaling	DA dependency
full_onchain_calldata	Highest configured cost at large payload	Direct payload availability	Linear on-chain byte growth	None
compact_external_da	Lower on-chain footprint,	DA proof required	Payload cost moves to DA	External DA availability

	higher DA profile cost than other compact routes		profile	
compact_eip4844_like	Lower configured marginal data cost	DA/blob-like retrieval required	Improves with larger payload and batch	Blob-like data availability assumption
compact_modular_da_sampling	Lowest configured marginal data cost	DA sampling/profile retrieval required	Most favorable in high-payload, high-batch workload	Strongest DA dependency in this model

表 9 DA route tradeoff comparison

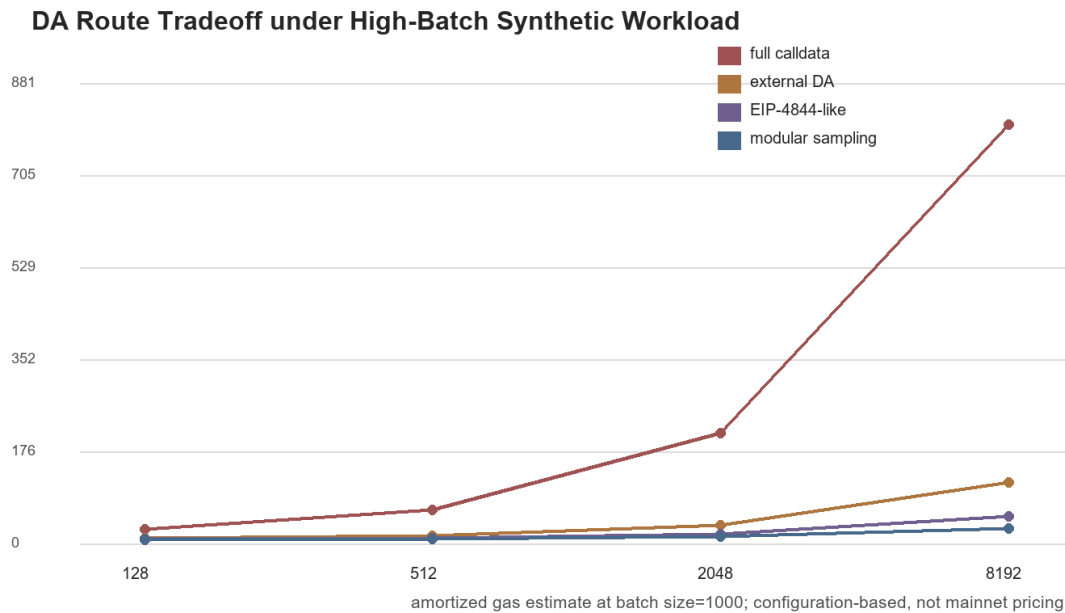


图 13 DA route tradeoff under a high-batch synthetic workload

Observation. Full calldata deteriorates as payload grows because full payload bytes remain chain-facing. Compact routes decouple the chain-facing commit from payload size, so their relative advantage grows under high-payload, high-batch workloads. The compact modular DA sampling profile is favorable in this configured estimate because it has the lowest marginal DA cost; the same result also means the design depends more strongly on a separate DA layer. The comparison supports a route-selection tradeoff, not a universal production-cost ranking.

7.7.5 Scalability Boundary

The current prototype does not produce a measured concurrent challenge benchmark. There is no output for multiple simultaneous challenge sessions, recovery queue growth, replay queue delay, or scheduler contention. For that reason, this revision does not add a production-style concurrent scalability claim. The existing `challenge_grid` still supports logarithmic bisection behavior over trace length; concurrent recovery and replay scheduling remain future work.

8 Discussion and Limitations

This discussion integrates limitations with the paper's claims. The value of the work is not a production-ready Rollup, but a scoped systems/security argument: recoverable challenge improves arbitration continuity, evidence-carrying compact commit preserves replay context, and DA-aware estimation clarifies cost boundaries for high-frequency workloads. simulated TEE, mock/verifiable DA registry, synthetic workloads, and local EVM baselines limit external validity, but they do not erase the mechanism-level question the paper studies.

8.1 Internal Validity（内部效度）

内部效度主要受 simulated TEE、mock/verifiable DA registry、Python 原型逻辑与 Hardhat 本地 EVM 测量方式影响。原型中的 `attestation` 只建模输入/输出绑定，DA 层也没有接入真实采样与经济安全机制，因此实验验证的是机制接口是否能跑通，而不是硬件安全或 DA 经济安全。本地 `gasUsed` 还可能受到合约简化、测试数据规模固定和本地链环境的影响。

8.2 External Validity（外部效度）

外部效度的限制来自实验环境到主网或生产网络之间的差距。主网 MEV、拥堵费率波动、blob 价格变化、真实 TEE 供应链攻击、侧信道风险、宿主 I/O 操控与真实 DA 网络运营风险，均未在当前实验中覆盖。Sepolia 部署只能说明核心合约具有公开测试网可部署性，不能据此外推主网性能或安全强度。

8.3 Construct Validity and Deployment Boundary（构念效度与部署边界）

构念效度关注指标与论文主张之间是否匹配。`challenge success`、`recovery_success` 与 `slashed_rate` 衡量的是原型状态机下的仲裁推进，而不是完整 Rollup 经济安全；DA-aware cost model 是配置化趋势模型，不等价于真实 `calldata/blob` 定价；`fault taxonomy` 也尚未形成统一 slashing 闭环。因此，论文结论应理解为机制层研究原型结论，而非生产系统审计结果。

具体而言，TEE 部分仍停留在模拟层。`attestation` 用于绑定输入、输出、nonce 和执行环境标识，目的在于验证“链下执行摘要 - 链上提交 - 异常挑战”的协议接口；真实 SGX、TDX、CSV

或其他硬件 TEE 还需要处理远程证明格式、硬件漏洞、侧信道攻击、宿主 I/O 操控和多厂商信任假设。

DA 部分同样是 mock/verifiable DA 原型。系统实现了 DA root、payload hash、字段级 Merkle proof 和可用性状态记录，但没有接入 Celestia、EigenDA 或其他真实 DA 网络。因而，compact commit 与 verifiable DA structure 在本文中只能说明可检查证据入口的形成方式，不能等同于生产级 DA availability。

成本模型也应按配置化估算理解。DA profile 中的 gas per byte 参数用于构造可比较的趋势模型，而不是主网 calldata、blob 或真实 DA 服务的实时价格。由此得到的 96.33% 降本只对应给定 cost model 下的趋势结果，不能外推为主网级或生产级费用结论。

Hardhat gasUsed 提供的是本地 EVM 基线。它说明 Solidity 原型中的 submit、challenge open、recover、replay、resolve 和 finalize 等关键路径具有链上状态推进对应物，并能被本地测试链测量；但它不等价于 Sepolia 或主网费用，也没有覆盖公网拥堵、交易排序、blob 价格和跨合约部署环境差异。

Sepolia 部署记录属于公开测试网 deployability evidence。部署区块、合约地址和 ABI 索引说明当前合约可以部署到公开测试网；同时，源码验证曾受 block explorer connect timeout 影响，也尚未记录大规模 Sepolia 交互交易样本。该证据不应被解释为生产成熟度、主网性能或完整安全性的证明。

fault taxonomy 仍需继续收敛。当前 response_tampered、trace_length_mismatch 和 challenge_timeout_recover 可以进入 challenge + slashing 路径；attestation_invalid、da_unavailable 和 da_proof_invalid 主要体现为检测、拒绝或报告能力。后续需要明确不同 fault 的仲裁策略，例如哪些故障应直接拒绝，哪些故障需要 watchdog 恢复，哪些故障应进入链上仲裁或经济惩罚。

challenge replay 中的部分 evidence 仍由链下提供。原型通过 expected hash、claimed hash、trace evidence 和 mismatch bits 支持 replay arbitration，但没有实现完整链上可验证执行环境、FPVM 或真实模型推理单步证明。下一步需要加强 evidence 的链上可验证性、形式化安全论证和跨实现一致性测试，减少 replay 结果对链下诚实生成证据的依赖。

Taken together, these boundaries define the next research steps rather than weakening the central mechanism claim. Future work should replace simulated attestation with real TEE evidence, connect to real DA networks, measure concurrent challenge queues, strengthen replay evidence verification, and formalize fault-to-slashing policy. The current contribution remains the protocol narrative and prototype evidence for failure -> recovery -> replay -> resolution under scoped assumptions.

8.4 Relationship to Prior Work

本文与现有工作的关系属于 mechanism enhancement，而非 system replacement。本文并不试图替代 TEERollup、OTR 或 opML 等系统路线，而是在其异常路径上补充 Recoverable Challenge 与 Evidence-Carrying DA Binding，使 Hybrid TEE-Rollup 在 timeout、DA 异常与轻量提交场景下具备更可解释的仲裁推进能力。

TEEROLLUP 提出了利用异构 TEE 降低 Rollup 验证成本并缩短提现延迟的系统设计，其核心包括 TEE 委员会、链上状态合约、挑战机制和数据可用性惩罚。本文受到 Hybrid TEE-Rollup 思路启发，但并不重做完整 TEEROLLUP 系统，而是聚焦异常场景下 challenge 的恢复性，以及 compact commit 与 verifiable DA 的成本和证据结构。换言之，本文的贡献位置不是替代既有 TEE-Rollup 架构，而是在其“TEE 快速路径 + 链上异常路径”的思想下，补充 timeout 后争议会话如何恢复推进、轻量提交如何保留 DA 证据入口、以及成本趋势如何以可复现实验方式表达。

OTR 和 Optimistic TEE-Rollups 关注如何将 TEE 与 optimistic verification 结合，尤其适用于链上生成式 AI 或模型推理结果验证。这类工作表明 TEE 可以作为快速路径，而 optimistic challenge 可作为异常路径。本文与其相近之处在于同样关注 TEE 与 challenge 的结合，差异在于本文强调 recoverable challenge 的 liveness，以及 DA payload 与 compact commit 之间的证据绑定。

opML 等工作研究如何通过 optimistic fraud proof 支持机器学习或大模型计算，通常包含二分定位、单步仲裁和链上虚拟机验证。本文借鉴交互式争议定位思想，但没有实现完整 FPVM 或真实 ML 执行证明，而是将 bisection replay 用于 Hybrid TEE-Rollup 原型中的异常仲裁。

Dynamic Fraud Proof 关注如何缩短无争议场景下的最终性，并在发现争议时动态延迟结算。该方向强调 challenge window 和验证者参与机制的动态调整。本文不提出完整动态最终性协议，而是在更小范围内研究 timeout 后 challenge session 如何恢复推进。

LazyLedger 和 Light Clients for Lazy Blockchains 等工作强调将数据可用性从执行验证中解耦，并通过数据可用性抽样支持模块化扩容。本文没有实现真实 DAS 网络，而是使用 mock DA 和 DA profile 建模 compact commit + verifiable DA 的成本与证据结构。本文的重点不是替代 DA 层，而是说明 Hybrid TEE-Rollup 中链上轻量提交必须与 DA 证明建立可验证联系。

EIP-4844 将 blob-carrying transaction 引入以太坊扩容路线，为 Rollup 数据发布提供了区别于传统 calldata 的成本结构。本文中的 compact EIP-4844-like profile 只抽象了“数据发布成本低于 full calldata”的趋势，并未模拟真实 blob base fee、blob 生命周期或主网交易调度。因此，EIP-4844 在本文中是成本建模参照，而不是已经接入的真实数据通道。

表 10 Related Work 与本文工作的对照

方向 / 工作	核心关注	与本文关系	本文不声称覆盖的部分
TEEROLLUP / Hybrid TEE-Rollup	利用异构 TEE 降低 Rollup 验证成本，并保留挑战和 DA 处罚机制	本文沿用 Hybrid TEE-Rollup 的研究方向，聚焦 recoverable challenge	不实现完整 TEE 委员会、生产级提现流程或完整经济安全

方向 / 工作	核心关注	与本文关系	本文不声称覆盖的部分
		与 evidence-carrying compact commit	
Optimistic TEE-Rollups / OTR	将 TEE 快速执行与 optimistic verification 结合	本文同样采用“正常路径低成本、异常路径挑战”的思想	不证明真实 TEE 远程证明安全或链上生成式 AI 生产服务可用性
opML / optimistic ML fraud proof	通过二分定位和单步证明验证 ML 或复杂计算	本文借鉴 bisection replay 的争议定位结构	不实现完整 FPVM、真实模型推理单步证明或链上 ML 执行环境
Dynamic Fraud Proof	动态调整争议窗口和最终性，以缩短无争议路径延迟	本文关注 timeout 后会话恢复，是更窄的 challenge liveness 问题	不提出完整动态最终性协议
LazyLedger / 模块化 DA	将 DA 与执行验证解耦，支持数据可用性抽样	本文用 mock/verifiable DA 与 DA profile 说明 compact commit 需要绑定 DA 证据	不接入真实 DA 网络，也不证明 DAS 经济安全
Light Clients for Lazy Blockchains	面向轻客户端的数据可用性抽样与验证	本文借鉴“轻量验证者需要可检查数据承诺”的思想	不实现真实轻客户端协议
EIP-4844 / blob data	降低 Rollup 数据发布成本，提供 blob 数据市场	本文将 EIP-4844-like 作为配置化成本 profile	不模拟真实 blob base fee、主网拥堵或 blob 生命周期

从对照可以看出，已有工作已经提供 TEE-Rollup、optimistic fraud proof 和 modular DA 的主要路线；本文的课程论文工作把 recoverable challenge、DA-bound compact commit 和轻量成本模型放入同一个可运行研究原型中，并分别用 Python、Hardhat 与 Sepolia 证据说明机制可行性、链上对应物和可部署性边界。

8.5 Takeaways and Future Direction

高频 DApp 的核心困难不只是吞吐量不足，还在于低成本交互与异常可验证之间经常互相拉扯。完全上链会把大量低价值高频请求转化为高昂链上成本；完全链下或单纯依赖 TEE 又会削弱异常时的公开仲裁能力。本文在已有 Hybrid TEE-Rollup 思路下回答的是一个更窄的问题：能否在不重做完整 Rollup 系统的前提下，让轻量提交、DA 证据和可恢复 challenge 形成可运行的研究原型。

The evidence gives a cautious but useful answer. The Python prototype exercises commit, DA proof, challenge, recovery, replay, and resolution; the comparative evaluation separates detection from completion and compression from evidence linkage; the local EVM and Sepolia artifacts show that the protocol path has a contract counterpart. This is enough to support a systems research claim about mechanism design under scoped assumptions, but not enough to claim production security or performance.

后续研究可以沿两个方向推进：一是把 evidence 的链上可验证性做得更强，减少 replay 对链下证据生成的依赖；二是在真实 DA 与 TEE 环境中重新测量成本、活性和异常处理边界。这

样, Hybrid TEE-Rollup 面向高频 DApp 的可验证低成本交互, 才可能从研究原型继续走向更完整的系统评估。

8.5.1 Theoretical Implication (理论启示)

从理论层面看, 本文更有价值的部分不是提出新的 Rollup 系统, 而是把 Challenge Recovery 从工程补丁提升为具有协议语义的独立机制。Recoverable Challenge、Evidence-Carrying Commit 与 DA-Aware Cost Model 的组合, 为 Hybrid TEE-Rollup 在高频 DApp 场景下的异常处理与低成本验证提供了一个可讨论的研究视角。

参考文献

- [8] Intel Corporation. Intel Software Guard Extensions (Intel SGX) Developer Guide[EB/OL]. 2024.
- [9] Intel Corporation. Intel Trust Domain Extensions (Intel TDX) Architecture Specification[EB/OL]. 2024.
- [10] Offchain Labs. Arbitrum: A Next-Generation Layer 2 for Ethereum[EB/OL]. 2021.
- [11] Cartesi. Cartesi Rollups Documentation[EB/OL]. 2024.
- [12] EigenLayer. EigenDA Documentation[EB/OL]. 2024.
- [13] Celestia. Celestia: Modular Blockchain Network[EB/OL]. 2024.
- [14] RISC Zero. RISC Zero zkVM Documentation[EB/OL]. 2024.
- [15] Succinct Labs. SP1: A Performant, 100% Open-Source zkVM[EB/OL]. 2024.
- [16] Arbitrum Foundation. Arbitrum Nitro Technical Documentation[EB/OL]. 2024.
- [17] OP Labs. Optimism Documentation: Fault Proofs and Dispute Games[EB/OL]. 2024.
- [18] Canetti R, et al. Universally Composable Security: A New Paradigm for Cryptographic Protocols[EB/OL]. 2000.
- [19] Goldwasser S, Micali S, Rackoff R. The Knowledge Complexity of Interactive Proof Systems[J]. SIAM Journal on Computing, 1989, 18(1): 186-208.
- [20] NIST. Recommendation for Key Management: Part 1 - General (Rev. 5)[EB/OL]. 2020.
- [1] WEN X, FENG Q, LYU H, et al. TEEROLLUP: Efficient Rollup Design Using Heterogeneous TEE[EB/OL]. arXiv:2409.14647, 2024.
- [2] PICCO G, FORTUGNO A. Dynamic Fraud Proof[EB/OL]. arXiv:2502.10321, 2025.
- [3] AL-BASSAM M. LazyLedger: A Distributed Data Availability Ledger with Client-Side Smart Contracts[EB/OL]. arXiv:1905.09274, 2019.
- [4] TAS E N, TSE D, YANG L, et al. Light Clients for Lazy Blockchains[EB/OL]. arXiv:2203.15968, 2022.
- [5] Ethereum Foundation. EIP-4844: Shard Blob Transactions[EB/OL]. 2024.
- [6] Ethereum Foundation. Optimistic Rollups[EB/OL]. Ethereum Documentation, 2024.
- [7] Buterin V. An Incomplete Guide to Rollups[EB/OL]. 2021.